ACTUALIZACIÓN TECNOLÓGICA · DESARROLLO DE APLICACIONES MÓVILES



Flutter

Un solo código para Android, iOS, web y escritorio



Jairo Andrés Zubieta Bermudez

Ingeniería de Sistemas · Universidad del Tolima · Tutoría IV

AGENDA

Lo que veremos en 15 minutos

01	Reseña histórica	2 min
02	Principales características	3 min
03	Proyectos reconocidos	2 min
04	Guía de instalación y uso	3 min
05	Ejemplo práctico en vivo	5 min

01 · RESEÑA HISTÓRICA

De un experimento de Google a estándar multiplataforma

2015

Nace como «Sky» en el Dart Developer Summit: dibujar a 120 fps en Android

2017

Primera versión alfa, presentada en Google I/O

2018

Flutter 1.0 estable en el evento Flutter Live (diciembre)

2021

Flutter 2: web estable y null safety en Dart

2022

Flutter 3: estable en 6 plataformas, incluidos Windows, macOS y Linux

2024

Motor **Impeller** en móviles y WebAssembly para web (tras Dart 3, 2023)

2026

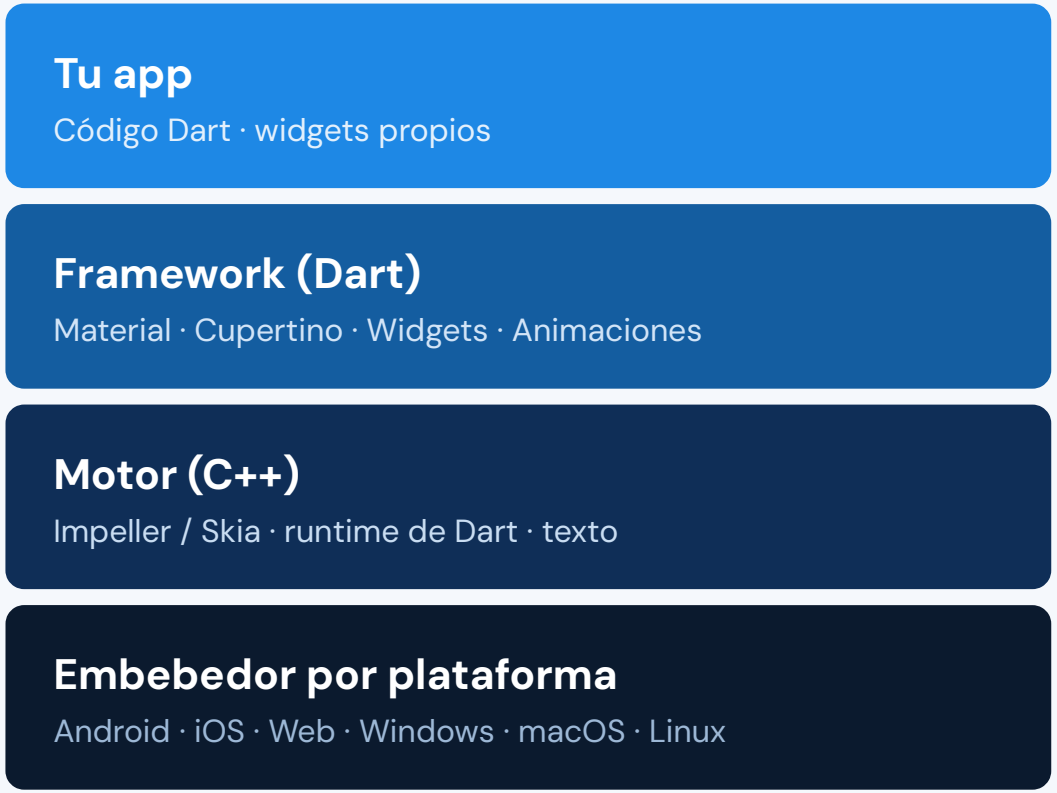
3.47 estable (agosto); Impeller llega también a escritorio

02 · ¿QUÉ ES FLUTTER?

Un kit de UI que dibuja cada píxel por su cuenta

Framework **open source** de Google, escrito en **Dart**. No usa los componentes nativos: trae su propio motor gráfico, por eso la app se ve igual en todas las plataformas.

Todo es un **widget**: botones, textos, márgenes y hasta la app completa.



Dart es compilado, pero de dos formas

MIENTRAS DESARROLLAS

JIT · Just In Time

Compila sobre la marcha. Por eso existe el **hot reload**: ves los cambios al instante.

CUANDO PUBLICAS

AOT · Ahead Of Time

Compila todo a **código máquina** antes de instalar: la app final es rápida, como una nativa.

Orientado a objetos

Clases y herencia, parecido a Java o C#.

Null safety

El compilador evita errores por valores nulos.

Creado por Google

Presentado en 2011; es la base de Flutter.

02 · WIDGETS, MATERIAL Y CUPERTINO

Todo en Flutter es un widget

¿Qué es un widget?

Una pieza de la interfaz: un texto, un botón, un margen o una pantalla entera. Los widgets se anidan unos dentro de otros y forman un **árbol**, como piezas de Lego.

Stateless

No cambia después de dibujarse.
Ej.: **Text**, **Icon**

Stateful

Tiene datos que cambian y se redibuja.
Ej.: **Checkbox**, nuestra lista

Material

El sistema de diseño de **Google**, típico de Android. Es el estilo por defecto en Flutter.

`MaterialApp` `AppBar` `ElevatedButton`

Cupertino

Imita el diseño de **Apple** (iOS). Su nombre viene de la ciudad sede de Apple.

`CupertinoApp` `CupertinoButton` `CupertinoSwitch`

02 · PRINCIPALES CARACTERÍSTICAS

Seis razones por las que se adopta

Un solo código

Android, iOS, web, Windows, macOS y Linux desde el mismo proyecto.

Hot reload

Aplica los cambios del código al instante, **sin reiniciar la app ni perder los datos** en pantalla.

Rendimiento nativo

Dart se compila (AOT) a código máquina; el motor apunta a 60–120 fps.

UI declarativa

Describes **cómo debe verse** la pantalla según los datos y Flutter la redibuja sola.

Paquetes en pub.dev

pub.dev es el repositorio oficial de librerías listas: Firebase, mapas, cámara, HTTP…

Open source y gratis

Licencia BSD, respaldo de Google y una comunidad muy activa.

03 · PROYECTOS RECONOCIDOS

Empresas que confían en Flutter

Google Pay

Reescrita en Flutter para unificar sus apps de Android e iOS.

Ver caso →

Nubank

Banco digital latinoamericano (Brasil, México, Colombia); su app móvil usa Flutter.

Ver caso →

BMW

App «My BMW» para controlar el vehículo, en decenas de países.

Ver caso →

Alibaba

Xianyu, su mercado de segunda mano, fue de los primeros grandes casos.

Ver caso →

eBay Motors

Compra y venta de vehículos; reutiliza casi todo el código entre plataformas.

Ver caso →

Toyota

Usa Flutter en sistemas de infoentretenimiento de sus vehículos.

Ver caso →

04 · GUÍA DE INSTALACIÓN (WINDOWS)

Cinco pasos hasta el primer «flutter doctor»

1
Herramientas base

Instalar Git y VS Code con la extensión «Flutter».

2
SDK de Flutter

Descargarlo desde VS Code o el zip de docs.flutter.dev en C:\src\flutter.

3
Variable PATH

Agregar la carpeta flutter\bin al PATH del usuario.

4
Android Studio

Android SDK, command-line tools y un emulador (AVD).

5
Verificar

Aceptar licencias y revisar que todo salga en verde.

```
> flutter doctor --android-licenses
> flutter doctor
```

04 · USO BÁSICO

Del comando al emulador

```
# crear y entrar al proyecto
> flutter create mis_tareas
> cd mis_tareas
# ver dispositivos y ejecutar
> flutter devices
> flutter run
# agregar un paquete de pub.dev
> flutter pub add http
# generar el instalador Android
> flutter build apk --release
```

Mientras corre la app

r → hot reload (conserva el estado)
R → hot restart (reinicia)
q → salir

Estructura del proyecto

lib/main.dart → nuestro código
pubspec.yaml → dependencias
android/ ios/ web/ → cada plataforma

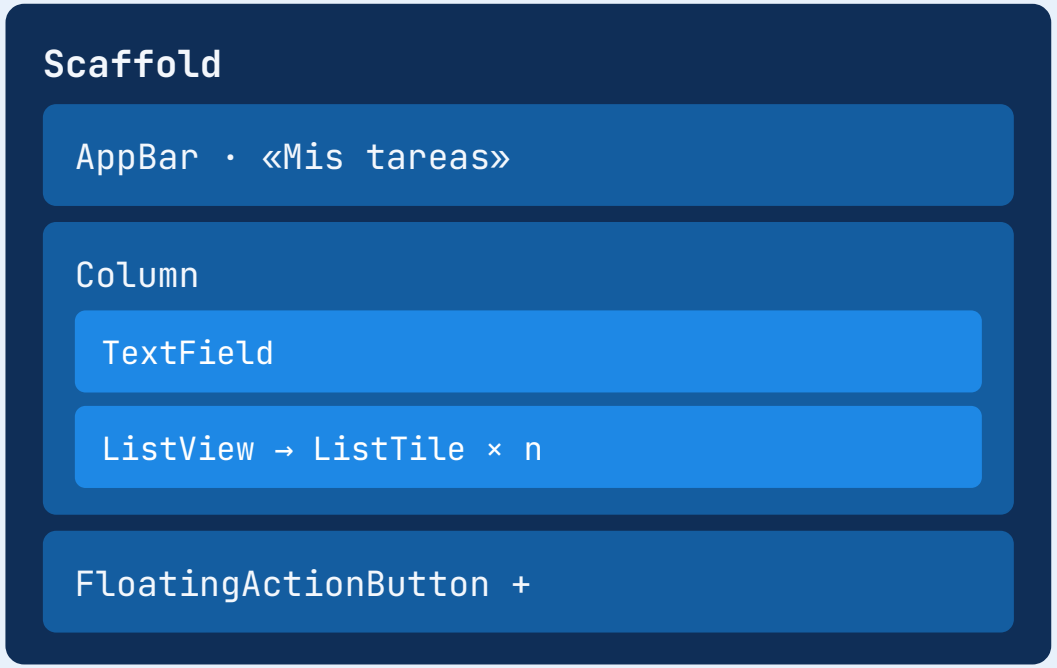
05 · EJEMPLO PRÁCTICO

«Mis tareas»: una app en menos de 40 líneas

Escribimos una tarea, la agregamos con el botón **+** y aparece en la lista. Un solo archivo, **lib/main.dart**.

Conceptos: **StatefulWidget**, **setState** y **hot reload**.

ÁRBOL DE WIDGETS



Scaffold *(«andamio») es el molde de una pantalla: tiene huecos para la barra, el cuerpo y el botón flotante.*

05 · CÓDIGO 1/2

El estado: guardar y agregar tareas

```dart
import 'package:flutter/material.dart';

void main() ⇒ runApp(const MaterialApp(home: Tareas()));

class Tareas extends StatefulWidget {
  const Tareas({super.key});
  @override
  State<Tareas> createState() ⇒ _TareasState();
}

class _TareasState extends State<Tareas> {
  final _items = <String>[];
  final _ctrl = TextEditingController();

  void _agregar() {
    if (_ctrl.text.trim().isEmpty) return;
    setState(() ⇒ _items.add(_ctrl.text.trim()));
    _ctrl.clear();
  }
```

main()

Arranca la app con MaterialApp y nuestra pantalla.

StatefulWidget

Pantalla con datos que cambian: la lista.

setState()

Avisa a Flutter que redibuje la interfaz.

05 · CÓDIGO 2/2

La interfaz: el método build()

```
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Mis tareas')),
      body: Column(children: [
        TextField(controller: _ctrl, onSubmitted: (_) ⇒ _agregar()),
        Expanded(
          child: ListView.builder(
            itemCount: _items.length,
            itemBuilder: (_, i) ⇒ ListTile(title: Text(_items[i])),
          ),
        ),
      ]),
      floatingActionButton: FloatingActionButton(
        onPressed: _agregar, child: const Icon(Icons.add)),
    );
  }
}
```

Demo en vivo

1. flutter run
2. Agregar 2–3 tareas
3. Cambiar el título y pulsar **r**: hot reload sin perder la lista

BALANCE

¿Cuándo elegir Flutter?

Ventajas

- Un equipo y un código para varias plataformas
- Desarrollo rápido gracias al hot reload
- Interfaz idéntica y fluida en todos lados
- Gratuito, documentado y con gran comunidad

Desventajas

- Apps más pesadas que una nativa mínima
- Hay que aprender Dart
- APIs nativas muy nuevas pueden requerir plugins o código nativo
- En web no es ideal para sitios que dependen del SEO

GRACIAS

¿Preguntas?



Documentación
docs.flutter.dev

Probar sin instalar
dartpad.dev

Paquetes
pub.dev

Casos de éxito
flutter.dev/showcase